

Universidad de San Carlos de Guatemala
Facultad de Ingenieria
Manejo e Implementacion de Archivos

Manual Tecnico

Proyecto 2 - Sistema de archivos EXT2/EXT3

Carlos Eduardo Díaz Chacón

Carne 202400245

Segundo Semestre 2026

Indice

Indice.....	
1. Introduccion.....	
2. Arquitectura general.....	
3. Estructura del proyecto.....	
4. Estructuras de datos.....	
4.1 Tamaños en disco.....	
4.2 Apuntadores de un inodo.....	
5. Distribucion de la particion.....	
5.1 Calculo del numero de inodos.....	
5.2 Diferencia entre EXT2 y EXT3.....	
5.3 Registro en la bitacora.....	
6. Analisis de comandos.....	
6.1 Resolucion de simbolos.....	
6.2 Flujo de ejecucion.....	
6.3 Tabla de despacho.....	
7. Escritura en disco.....	
8. Permisos.....	
9. API REST.....	
9.1 Politica de origen cruzado.....	
9.2 Manejo de errores.....	
9.3 Separacion entre validar y modificar.....	
10. Interfaz web.....	
10.1 Origen del estado de sesion.....	
10.2 Acceso escalonado.....	
10.3 Consultas concurrentes.....	
11. Compilacion.....	
12. Despliegue.....	
12.1 Ejecucion permanente.....	
12.2 Direccion del servidor.....	

1. Introduccion

Este documento describe la arquitectura interna, las estructuras de datos y las decisiones de diseño del sistema desarrollado para el Proyecto 2 del curso Manejo e Implementación de Archivos. Está dirigido a quien necesite comprender, mantener o extender el código fuente.

El sistema simula un sistema de archivos EXT2 y EXT3 completo dentro de archivos binarios. La totalidad de la información persiste en disco: no existe ninguna estructura del sistema de archivos que se mantenga únicamente en memoria.

2. Arquitectura general

El proyecto se divide en tres capas que se comunican mediante interfaces bien definidas:

Capa	Tecnología	Responsabilidad
Presentación	React y Vite	Interfaz gráfica: terminal, autenticación y visualizador.
Servicio	cpp-httplib y nlohmann/json	Expone la lógica del sistema como una API REST sobre HTTP.
Dominio	C++17 y ANTLR4	Análisis de comandos y manipulación del sistema de archivos simulado.

La separación permite que el mismo núcleo en C++ sea utilizado por dos programas distintos: un ejecutable de consola que procesa archivos de script, y un servidor HTTP que atiende a la interfaz web. Ambos comparten los mismos objetos compilados y por lo tanto el mismo comportamiento.

3. Estructura del proyecto

```

MIA_2S2026_P2_202400245/
|-- backend/
|   |-- grammar/          Gramaticas de ANTLR
|   |-- generated/       Analizadores generados
|   |-- src/
|       |-- estructuras/  Structs que se escriben en disco
|       |-- disco/       Lectura y escritura del sistema de archivos
|       |-- comandos/    Implementacion de cada comando
|       |-- reportes/    Generacion de reportes con Graphviz
|       |-- util/        Utilidades de texto y archivos
|       |-- Analizador.cpp  Puente entre ANTLR y el ejecutor
|       |-- Ejecutor.cpp   Tabla de despacho de comandos
|       |-- Api.cpp       Endpoints en formato JSON
|       |-- main.cpp      Punto de entrada de la consola
|       |-- servidor.cpp  Punto de entrada del servidor HTTP
|   |-- compilar.sh      Script de compilacion
+-- frontend/
    |-- src/
        |-- api.js        Capa de comunicacion con el backend
        |-- App.jsx       Armazon y estado de sesion
        |-- Terminal.jsx  Entrada y salida de comandos
        |-- Login.jsx     Formulario de autenticacion
        |-- Visualizador.jsx Explorador del sistema de archivos

```

4. Estructuras de datos

Todas las estructuras que se escriben en disco están declaradas en un único archivo de cabecera y se encuentran encerradas entre directivas de empaquetado:

```
#pragma pack(push, 1)
// declaracion de todas las estructuras
#pragma pack(pop)
```

Esta directiva elimina el relleno que el compilador introduce por defecto para alinear los campos en memoria. Sin ella, la estructura Inodo ocuparía 104 bytes en lugar de los 100 que corresponden a la suma exacta de sus campos, y la distribución del disco dejaría de coincidir con la especificada. Para garantizar que el tamaño nunca cambie de forma accidental, el archivo incluye verificaciones en tiempo de compilación:

```
static_assert(sizeof(Information) == 110, "information debe medir 110");
static_assert(sizeof(Journal) == 114, "journal debe medir 114");
static_assert(sizeof(BloqueCarpeta) == 64, "bloque carpeta debe medir 64");
```

Si alguien modifica un campo de estas estructuras, la compilación falla de inmediato en lugar de producir discos corruptos que serían difíciles de diagnosticar.

4.1 Tamaños en disco

Estructura	Tamaño	Propósito
Particion	35 bytes	Describe una partición dentro del MBR.
MBR	157 bytes	Cabecera del disco con sus cuatro particiones.
EBR	30 bytes	Cabecera de una partición lógica.
SuperBloque	76 bytes	Metadatos del sistema de archivos.
Inodo	100 bytes	Describe un archivo o una carpeta.
Contenido	16 bytes	Una entrada dentro de un bloque de carpeta.
BloqueCarpeta	64 bytes	Cuatro entradas de carpeta.
BloqueArchivo	64 bytes	Sesenta y cuatro caracteres de contenido.
BloqueApuntadores	64 bytes	Dieciséis apuntadores a otros bloques.

Estructura	Tamano	Proposito
Information	110 bytes	Datos de una operacion registrada.
Journal	114 bytes	Una entrada de la bitacora de EXT3.

Los tres tipos de bloque miden exactamente lo mismo, lo que permite tratarlos de manera uniforme al calcular posiciones dentro del disco. El tamaño de bloque del sistema se define a partir de esa equivalencia.

4.2 Apuntadores de un inodo

Cada inodo contiene un arreglo de quince apuntadores a bloques, distribuidos de la siguiente forma:

Posiciones	Tipo	Bloques alcanzables
0 a 11	Directos	12
12	Indirecto simple	16
13	Indirecto doble	256
14	Indirecto triple	4096

La capacidad máxima de un archivo es de 4380 bloques, equivalentes a 280 kilobytes aproximadamente. Un bloque de apuntadores contiene dieciseis enteros, de donde provienen las cantidades de los niveles indirectos.

```
js  Sesion.h  Sesion.cpp  Api.cpp  Login.jsx  Manual_Tecnico_202400245.docx
backend > src > disco > Ext2.cpp > inicioJournal(const SuperBloque &)
143
144 int bloqueLogico(const std::string &ruta, SuperBloque &sb, Inodo &nodo,
145                 int n, bool asignar) {
146     // directos
147     if (n < 12) {
148         if (nodo.i_block[n] == -1) {
149             if (!asignar) return -1;
150             nodo.i_block[n] = asignarBloque(ruta, sb);
151         }
152         return nodo.i_block[n];
153     }
154     n -= 12;
155
156     // simple indirecto inodo -> apuntadores -> datos
157     if (n < PUNTEROS) {
158         if (!prepararIndirecto(ruta, sb, nodo, 12, asignar)) return -1;
159         return puntero(ruta, sb, nodo.i_block[12], n, asignar, false);
160     }
161     n -= PUNTEROS;
162
163     // doble indirecto inodo -> apuntadores -> apuntadores -> datos
164     if (n < PUNTEROS * PUNTEROS) {
165         if (!prepararIndirecto(ruta, sb, nodo, 13, asignar)) return -1;
166     }
```

Figura 2. Fragmento del código que recorre los apuntadores indirectos

5. Distribucion de la particion

Al formatear una particion se calcula cuantos inodos caben en el espacio disponible. La formula parte de la relacion establecida en la especificacion: por cada inodo se reservan tres bloques.

5.1 Calculo del numero de inodos

```
int disponible = m->tam - sizeof(SuperBloque);
if (ext3) disponible -= JOURNAL_ENTRADAS * sizeof(Journal);
int n = disponible / denominador;
```

El denominador se compone de la suma de todo lo que consume un inodo dentro de la particion:

Concepto	Bytes
Entrada en el mapa de bits de inodos	1
Entradas en el mapa de bits de bloques (tres por inodo)	3
Tamano del inodo	100
Tamano de los tres bloques asociados	192
Total por inodo	296

5.2 Diferencia entre EXT2 y EXT3

La unica diferencia estructural entre ambos sistemas es la presencia de la bitacora, que se ubica inmediatamente despues del superbloque y antes del mapa de bits de inodos. Ocupa un espacio fijo de cincuenta entradas.

Region	EXT2	EXT3
SuperBloque	offset 0	offset 0
Bitacora	no existe	offset 76, 5700 bytes
Mapa de bits de inodos	offset 76	offset 5776
Mapa de bits de bloques	a continuacion	a continuacion

Region	EXT2	EXT3
Tabla de inodos	a continuacion	a continuacion
Tabla de bloques	a continuacion	a continuacion

Dado que el superbloque almacena la posición absoluta de cada región, el resto del código funciona sin modificaciones para ambos sistemas. El único punto que requiere distinguirlos es el cálculo inverso que obtiene el inicio de la partición a partir del superbloque:

```
int inicioParticion(const SuperBloque &sb) {
    int atras = sizeof(SuperBloque);
    if (sb.s_filesystem_type == 3)
        atras += JOURNAL_ENTRADAS * sizeof(Journal);
    return sb.s_bm_inode_start - atras;
}
```

El campo `s_filesystem_type` almacena el valor 2 o 3 según el sistema con el que se formateo, lo que permite identificar el tipo leyendo únicamente el superbloque.

5.3 Registro en la bitacora

Cada operación que modifica el sistema de archivos agrega una entrada a la bitacora. La función encargada localiza la primera entrada libre, identificada por tener el contador en cero, y escribe allí la información de la operación. Si la partición no es EXT3 la función retorna sin hacer nada, de modo que los comandos no necesitan comprobar el tipo de sistema antes de invocarla.

```

backend > src > disco > Journal.cpp > ...
1  #include "Journal.h"
2  /
3  #include <ctime>
4
5
6
7
8
9  void anotarJournal(const Montaje *m, const std::string &operacion,
10                    const std::string &ruta, const std::string &contenido) {
11      if (m == nullptr) return;
12
13      SuperBloque sb;
14      if (!leerSB(m->ruta, m->inicio, sb)) return;
15      if (!esExt3(sb)) return;          // ext2 no lleva bitacora
16
17      long base = inicioJournal(sb);
18
19      // la primera entrada con j_count en cero es la que esta libre
20      for (int i = 0; i < JOURNAL_ENTRADAS; ++i) {
21          long pos = base + i * static_cast<long>(sizeof(Journal));
22
23          Journal entrada;
24          if (!leerDe(m->ruta, pos, entrada)) return;
25          if (entrada.j_count != 0) continue;
26
27          std::memset(&entrada, 0, sizeof(Journal));
28          entrada.j_count = i + 1;
29          copiarCampo(entrada.j_content.i_operation, 10, operacion);

```

Figura 4. Funcion que registra una operacion en la bitacora

6. Analisis de comandos

El procesamiento de la entrada se realiza con ANTLR4, que genera a partir de dos gramaticas el analizador lexico y el sintactico.

Archivo	Funcion
Ext2Lexer.g4	Define los simbolos del lenguaje: nombres de comando, parametros, valores y rutas.
Ext2Parser.g4	Define la estructura valida de cada comando y los parametros que admite.

6.1 Resolucion de simbolos

El analizador lexico resuelve las ambigüedades mediante dos reglas: ante dos coincidencias posibles prefiere la mas larga, y ante dos de igual longitud prefiere la declarada primero. Por esa razon el orden de las declaraciones es significativo y los simbolos especificos deben aparecer antes que los generales.

Un ejemplo concreto es el simbolo que reconoce patrones con comodines, que debe declararse antes del identificador generico para que una expresion como asterisco punto txt no sea interpretada como un nombre comun.

6.2 Flujo de ejecucion

El recorrido completo de un comando, desde que el usuario lo escribe hasta que se modifica el archivo en disco, atraviesa las siguientes etapas:

1. La interfaz envia el texto completo mediante una peticion al endpoint de ejecucion.
2. El analizador lexico convierte el texto en una secuencia de simbolos.
3. El analizador sintactico valida la estructura y construye el arbol de derivacion.
4. El analizador recorre el arbol y produce una estructura con el nombre del comando y sus parametros.
5. El validador comprueba que los parametros obligatorios esten presentes y que los valores sean coherentes.

6. El ejecutor busca el comando en la tabla de despacho e invoca la funcion correspondiente.
7. La funcion opera sobre el archivo mediante posicionamiento y escritura directa de estructuras.
8. Si la particion es EXT3, la operacion se registra en la bitacora.
9. El resultado se devuelve como una lista de lineas que la interfaz muestra en el area de salida.

6.3 Tabla de despacho

El ejecutor asocia cada nombre de comando con la funcion que lo implementa mediante una tabla construida una sola vez. Esta solucion evita una cadena extensa de comparaciones y permite agregar comandos nuevos con una sola linea.

```
{ "mkdisk", cmdMkdisk },  
{ "fdisk", cmdFdisk },  
{ "mount", cmdMount },  
{ "mkfs", cmdMkfs },  
{ "journaling", cmdJournaling },  
{ "loss", cmdLoss },
```

Todas las funciones comparten la misma firma, recibiendo los parametros ya validados y una referencia donde depositan el resultado. Esta uniformidad es lo que permite almacenarlas en una misma tabla.

```
backend > src > Ejecutor.cpp > {} 'anonymous-namespace' > manejadores()
19
20 namespace {
21
22
23 using Manejador = std::function<void(const Parametros &, Salida &);>;
24
25 const std::map<std::string, Manejador> &manejadores() {
26     static const std::map<std::string, Manejador> tabla = {
27         { "mkdisk", cmdMkdisk },
28         { "rmdisk", cmdRmdisk },
29         { "fdisk", cmdFdisk },
30         { "mount", cmdMount },
31         { "mounted", cmdMounted },
32         { "mkfs", cmdMkfs },
33         { "login", cmdLogin },
34         { "logout", cmdLogout },
35         { "mkgrp", cmdMkgrp },
36         { "rmgrp", cmdRmgrp },
37         { "mkusr", cmdMkusr },
38         { "rmusr", cmdRmusr },
39         { "chgrp", cmdChgrp },
40         { "cat", cmdCat },
41         { "mkfile", cmdMkfile },
42         { "mkdir", cmdMkdir },
```

Figura 5. Fragmento de la tabla de despacho de comandos

7. Escritura en disco

La persistencia se resuelve con dos operaciones elementales: posicionar el cursor del archivo en el byte deseado y volcar la estructura completa.

```
fseek(f, posicion, SEEK_SET);
fwrite(&dato, sizeof(T), 1, f);
```

Ambas operaciones están encapsuladas en funciones con plantilla, de modo que la misma implementación sirve para cualquiera de las estructuras del sistema. El compilador genera una versión especializada por cada tipo utilizado.

Es importante distinguir los dos sistemas de archivos que intervienen. El primero es el del sistema operativo, que almacena el archivo .mia como cualquier otro archivo. El segundo es el simulado, que existe únicamente dentro de ese archivo y es gestionado íntegramente por este proyecto. Las rutas que recibe el comando mkdisk pertenecen al primero, mientras que las rutas de comandos como mkdir pertenecen al segundo.

```
backend > src > util > Archivo.cpp > ...
6  bool existeArchivo(const std::string &ruta) {
7      struct stat info;
8      return stat(ruta.c_str(), &info) == 0 && S_ISREG(info.st_mode);
9  }
10
11  bool crearCarpetasPadre(const std::string &rutaArchivo) {
12      /* Recorre la ruta creando cada carpeta intermedia. Se detiene antes
13       * del ultimo '/' porque lo que sigue es el nombre del archivo. */
14      size_t ultimaBarra = rutaArchivo.find_last_of('/');
15      if (ultimaBarra == std::string::npos || ultimaBarra == 0) return true;
16
17      std::string carpetas = rutaArchivo.substr(0, ultimaBarra);
18
19      for (size_t i = 1; i <= carpetas.size(); ++i) {
20          if (i == carpetas.size() || carpetas[i] == '/') {
21              std::string parcial = carpetas.substr(0, i);
22              /* EEXIST no es un fallo: la carpeta ya estaba. */
23              if (mkdir(parcial.c_str(), 0755) != 0) {
24                  struct stat info;
25                  if (stat(parcial.c_str(), &info) != 0 || !S_ISDIR(info.st_mode))
26                      return false;
27              }
28          }
29      }
```

Figura 5. Funciones con plantilla para lectura y escritura de estructuras

8. Permisos

Los permisos se almacenan como tres caracteres que representan dígitos octales correspondientes al propietario, al grupo y al resto de usuarios. La verificación se realiza aplicando una máscara de bits sobre el dígito que corresponda según la relación entre el usuario de la sesión y el propietario del archivo.

Operacion	Mascara	Bit
Lectura	4	100
Escritura	2	010
Ejecucion	1	001

El usuario root omite la verificación de permisos, de acuerdo con el comportamiento habitual de los sistemas tipo Unix.

9. API REST

El servidor expone dos clases de endpoints. El primero recibe comandos en texto y devuelve lineas de salida destinadas a la terminal. Los restantes devuelven informacion estructurada en JSON para alimentar el visualizador.

Metodo	Ruta	Parametros	Respuesta
POST	/ejecutar	comandos	Lista de lineas de salida.
POST	/login	id, usuario, pass	Datos de la sesion iniciada.
POST	/logout	ninguno	Confirmacion del cierre.
GET	/sesion	ninguno	Estado de la sesion actual.
GET	/discos	ninguno	Discos registrados con sus metadatos.
GET	/particiones	disco	Particiones primarias, extendidas y logicas.
GET	/carpeta	id, path	Entradas de una carpeta.
GET	/archivo	id, path	Contenido y metadatos de un archivo.
GET	/journaling	id	Entradas de la bitacora.

9.1 Politica de origen cruzado

La interfaz web y el servidor se ejecutan en puertos distintos, lo que el navegador considera origenes diferentes. Para permitir la comunicacion, el servidor incluye las cabeceras correspondientes en todas sus respuestas y responde a las peticiones de sondeo que el navegador envia antes de cada peticion con cuerpo.

```
servidor.set_default_headers({
  { "Access-Control-Allow-Origin", "*" },
  { "Access-Control-Allow-Headers", "Content-Type" },
  { "Access-Control-Allow-Methods", "GET, POST, OPTIONS" }
});
```

9.2 Manejo de errores

Cuando una operacion no puede completarse por razones de logica, el servidor responde con codigo doscientos y un objeto que contiene un campo de error con el motivo. La peticion fue atendida correctamente; lo que fallo fue la operacion solicitada.

La capa de comunicacion del frontend convierte esa situacion en una excepcion, de manera que los componentes manejan con un unico bloque de captura los tres tipos de fallo posibles: ausencia de conexion, codigo de error HTTP y error de logica.

Situacion	Deteccion
El servidor no responde	Fallo de la peticion de red.
Codigo HTTP de error	Estado distinto de correcto.
Operacion rechazada	Presencia del campo de error en la respuesta.

9.3 Separacion entre validar y modificar

La autenticacion se implemento separando la verificacion de credenciales del cambio de estado. Una funcion independiente consulta el archivo de usuarios y construye una sesion en una variable local, sin tocar la sesion global.

Esta separacion resuelve un problema concreto: el formulario de la interfaz reemplaza la sesion anterior, por lo que debe cerrarla antes de abrir la nueva. Si el cierre ocurriera antes de validar, un intento fallido dejaria al usuario sin su sesion previa. Validando primero, un error de contrasena deja la sesion existente intacta.

```

backend > src > comandos > Sesion.cpp > cmdLogin(const Parametros &, Salida &)
31 // ----- LOGIN -----
32
33 // busca al usuario en users txt y deja la sesion armada en resultado
34 // no toca la global asi se puede validar sin perder la sesion que ya estaba
35 bool autenticar(const std::string &id, const std::string &usuario,
36                const std::string &pass, Sesion &resultado, std::string &motivo) {
37     std::string idMayus = aMayusculas(id);
38
39     const Montaje *m = buscarMontaje(idMayus);
40     if (m == nullptr) {
41         motivo = "no hay ninguna particion montada con id " + idMayus;
42         return false;
43     }
44
45     SuperBloque sb; int inodoUsers; Inodo nodo; std::string contenido;
46     if (!leerUsersTxt(m, sb, inodoUsers, nodo, contenido)) {
47         motivo = "la particion " + idMayus + " no esta formateada";
48         return false;
49     }
50
51     std::stringstream flujo(contenido);
52     std::string linea;
53     while (std::getline(flujo, linea)) {
54         std::vector<std::string> campos(linea);

```

Figura 7. Fragmento de la funcion de autenticacion

10. Interfaz web

La aplicación está construida con React y organizada en cuatro componentes, cada uno con una responsabilidad delimitada.

Componente	Responsabilidad
App	Mantiene el estado de la sesión, dibuja la cabecera y alterna entre secciones.
Terminal	Captura los comandos, los envía y muestra el resultado.
Login	Recoge las credenciales y reporta el resultado al componente que lo contiene.
Visualizador	Recorre discos, particiones, carpetas y archivos, y muestra la bitácora.

10.1 Origen del estado de sesión

La sesión reside en la memoria del proceso del servidor, no en el navegador. Por ello la aplicación consulta su estado al iniciarse, en lugar de asumir que no existe ninguna sesión. Sin esa consulta, recargar la página mostraría el formulario de autenticación mientras el servidor mantiene al usuario conectado.

Por la misma razón, la terminal notifica al componente principal cada vez que ejecuta comandos. Un script puede contener instrucciones de inicio o cierre de sesión, y la cabecera debe reflejar ese cambio.

10.2 Acceso escalonado

La terminal permanece disponible sin autenticación porque los discos y las particiones deben crearse antes de que exista cualquier usuario. El visualizador, en cambio, requiere sesión activa, ya que la lectura de archivos verifica los permisos contra el usuario conectado.

10.3 Consultas concurrentes

El visualizador cancela las respuestas que dejan de ser relevantes. Si el usuario navega entre carpetas con rapidez, una respuesta lenta podría llegar después de otra más

reciente y mostrar informacion que no corresponde a la ubicacion actual. La funcion de limpieza del efecto marca la peticion como obsoleta para que su resultado se descarte.

11. Compilacion

El script de compilacion realiza cuatro tareas en secuencia: genera los analizadores con ANTLR, compila los archivos generados, compila el codigo propio y enlaza los dos ejecutables.

El codigo compartido se compila una sola vez a archivos objeto, que luego se enlazan con cada uno de los dos puntos de entrada. De este modo se evita compilar dos veces las mismas unidades y se garantiza que ambos programas contengan exactamente la misma logica.

Etapa	Herramienta	Resultado
Generacion	ANTLR sobre la maquina virtual de Java	Analizadores en C++.
Compilacion	g++ con el estandar C++17	Archivos objeto.
Enlazado	g++ con las bibliotecas externas	Ejecutables consola y servidor.

El consumo maximo de memoria durante la compilacion corresponde al analizador sintactico generado y alcanza aproximadamente 287 megabytes, lo que permite compilar el proyecto en instancias de recursos limitados.

12. Despliegue

El servidor se ejecuta en una instancia de Amazon EC2 con sistema operativo Ubuntu. El acceso se realiza por SSH mediante un par de llaves, y el grupo de seguridad habilita unicamente los puertos necesarios.

Puerto	Protocolo	Proposito
22	TCP	Administracion remota por SSH.
8080	TCP	API REST consumida por la interfaz web.

12.1 Ejecucion permanente

Un proceso iniciado desde una sesión remota termina cuando esa sesión se cierra. Para evitarlo, el servidor se registra como un servicio del sistema, de modo que se inicia automáticamente al arrancar la instancia, sobrevive al cierre de la conexión y se reinicia si el proceso termina de forma inesperada.

Directiva	Efecto
After	Espera a que la red este disponible antes de iniciar.
User	Ejecuta el servicio con un usuario sin privilegios administrativos.
WorkingDirectory	Establece la carpeta desde la cual se resuelven las rutas relativas.
Restart	Vuelve a iniciar el proceso si termina de forma inesperada.
WantedBy	Inicia el servicio automáticamente al arrancar el sistema.

12.2 Dirección del servidor

La interfaz web obtiene la dirección del servidor desde una variable de entorno, con un valor por defecto que apunta al equipo local. Gracias a ello, el mismo código funciona en desarrollo y en producción sin modificaciones: basta definir la variable al construir la versión que se publica.

```
const URL_API = import.meta.env.VITE_API_URL ?? "http://localhost:8080";
```

La interfaz publicada debe servirse mediante HTTP y no HTTPS, ya que el navegador bloquea las peticiones hacia un servidor sin cifrado cuando la página se carga por un canal cifrado.